

PROJECT OBSIDIAN



PROJECT OBSIDIAN

Gabriel Henderson , Tyler Hollinger , Naiden
Maynard , Kyla Murray , Shalyn Croone

TABLE OF CONTENTS

Obsidian Cipher Design

How Substitution Works

How Permutation Works

The 16 Rounds

AI Integration

Live Demo

Technology Used

Lessons Learned

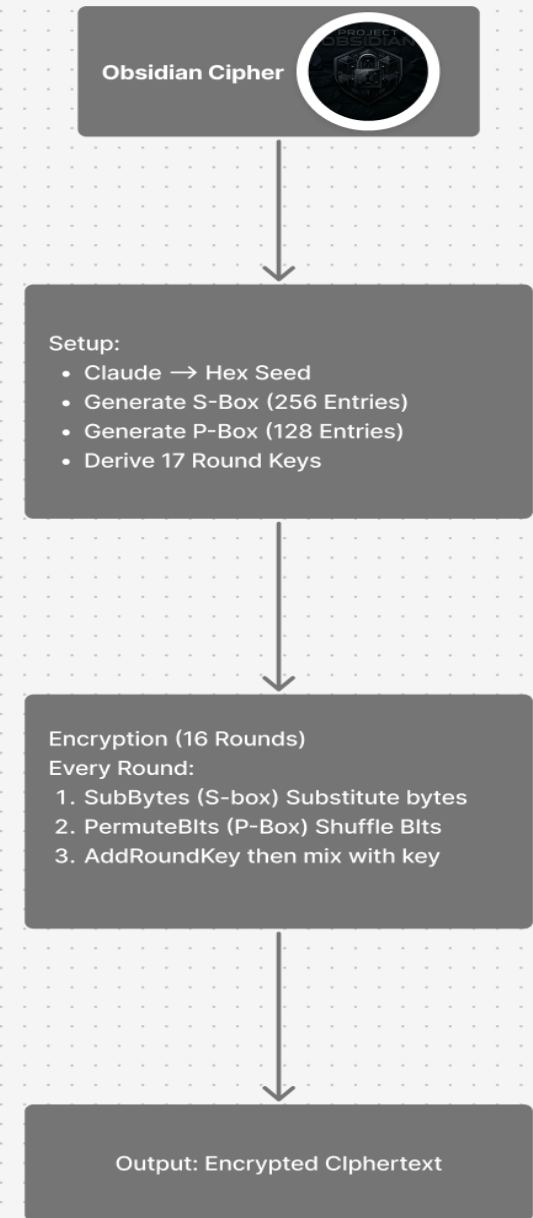
Questions ?



OBSIDIAN CIPHER DESIGN

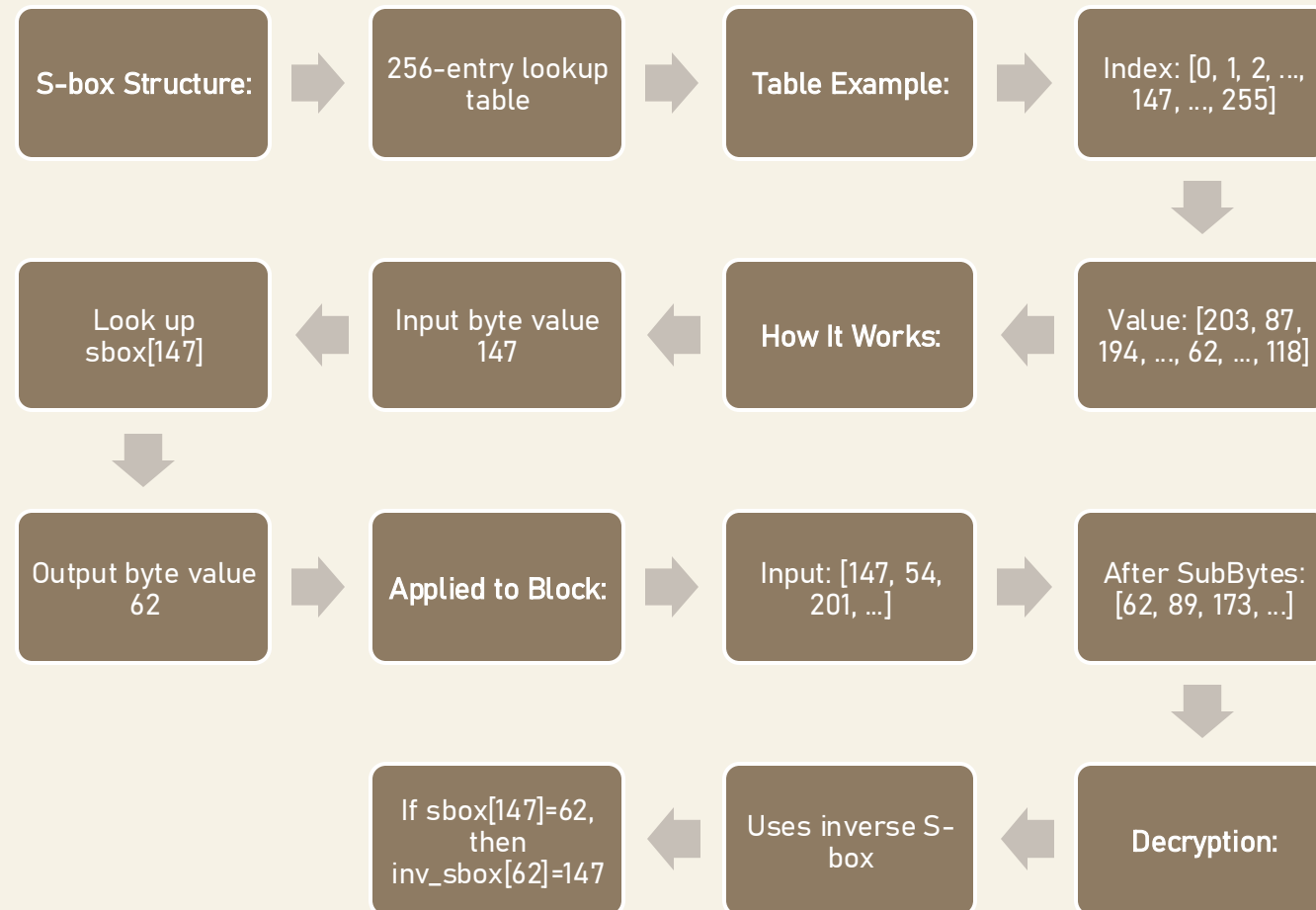
Claude generates a random seed once per session.

A 64-character hex string that serves as the source of randomness. Python feeds that seed into a Fisher-Yates shuffle to build two session-unique lookup tables: the S-box (256-entry byte substitution) and the P-box (128-entry bit permutation). Those tables, combined with a SHA-256 derived key schedule, drive 16 rounds of substitution, permutation, and XOR that transform your plaintext into ciphertext.



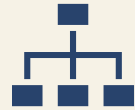
HOW SUBSTITUTION WORKS

The S-box is a 256-entry lookup table where each input byte gets replaced by its corresponding table value. For example, byte 147 becomes 62. For decryption, the inverse S-box reverses this by mapping 62 back to 147.



HOW PERMUTATION WORKS

The P-box converts the block to 128 individual bits and rearranges them. For example, output bit 0 comes from input bit 31, spreading changes across the entire block. Like the S-box, and we decrypt by using the inverse P-box.



P-box Structure:

128-entry position table



How It Works:

Convert 16 bytes $\rightarrow$ 128 bits

- $[b_0, b_1, b_2, \dots, b_{127}]$

Rearrange using P-box:

- $\text{output_bit}[i] = \text{input_bit}[\text{pbox}[i]]$

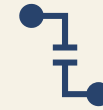
Example P-box: $[31, 0, 15, 47, \dots]$

- Output bit 0 $\leftarrow$ Input bit 31

- Output bit 1 $\leftarrow$ Input bit 0

- Output bit 2 $\leftarrow$ Input bit 15

Convert 128 bits $\rightarrow$ 16 bytes

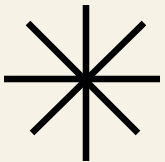
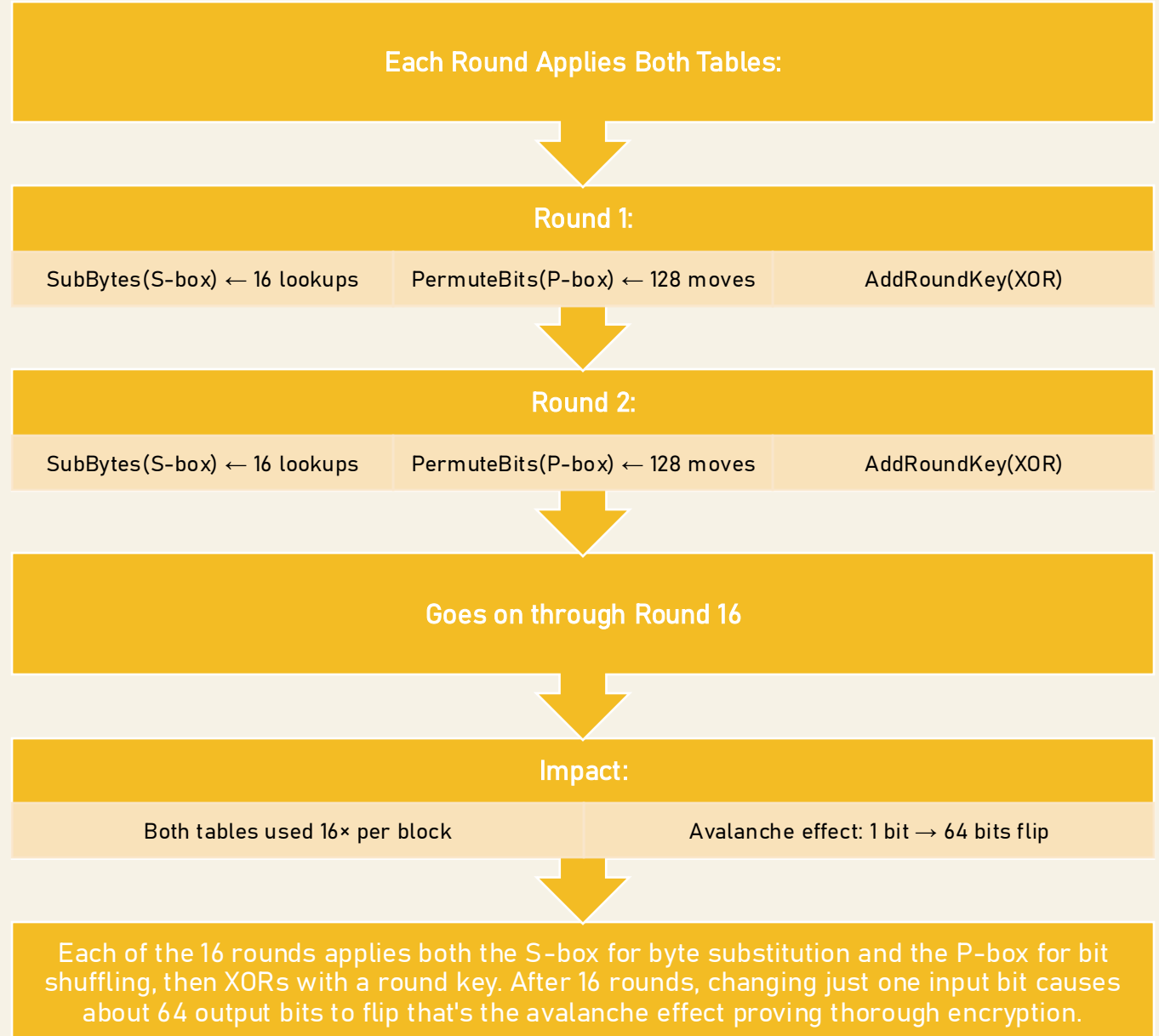


Result:

Every bit moves to new position

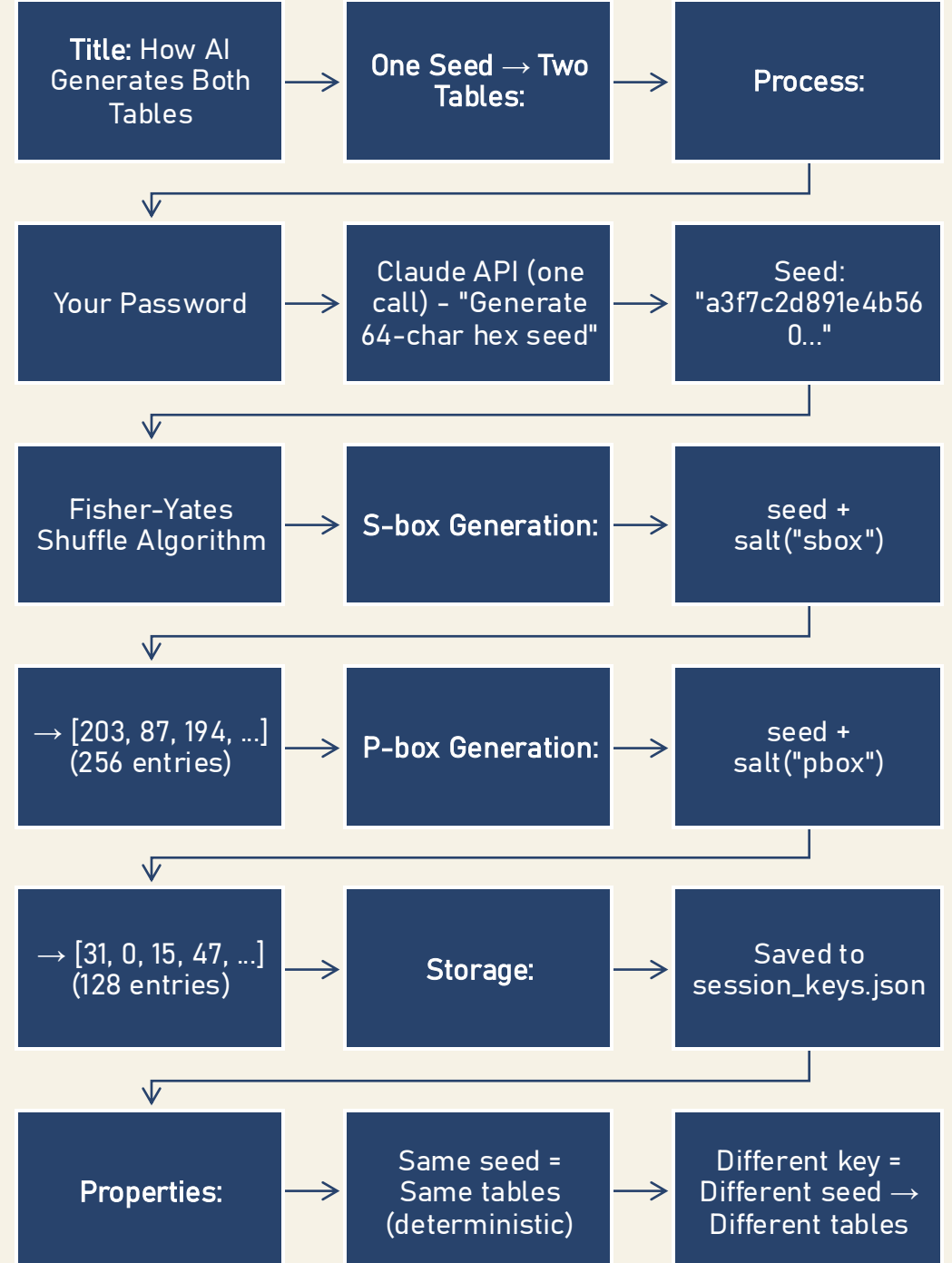
Decryption: Uses inverse P-box

THE 16 ROUNDS



AI INTEGRATION

One API call to Claude generates a hex seed, which is fed into Fisher-Yates shuffle twice with different salts to create both the 256-entry S-box and 128-entry P-box. Same password always produces the same tables for decryption, but different passwords create completely different tables encryption costs one API call, decryption is free from cache.



LIVE DEMO



TECHNOLOGY USED

Core Languages	The Logic (Custom)
• Python 3 (Backend Core)	• SPN Architecture (16 Rounds)
• HTML / CSS / JS (Frontend)	• Fisher-Yates Shuffle
• YAML / Markdown	• PKCS#7 Padding
• JSON (Session Config)	• XOR Round Key Mixing
AI & Integration	DevSecOps & Tools
• Anthropic Claude API	• Git / GitHub (Version Control)
• Model: Claude-4.6-Sonnet	• GitHub Actions (CI/CD)
• python-dotenv (Secret Mgmt)	• Pytest (Validation Suite)
• Flask (Local Web Server)	• VS Code (IDE)

LESSONS LEARNED



This project taught us three major things:

1. Deeper cryptography fundamentals. We learned why you must have true permutations with no duplicates, and that padding schemes like PKCS#7 are more complex than they seem.
2. AI integration reality: The biggest lesson was that LLMs can't reliably enumerate. Asking Claude to list 256 values failed 40% of the time. The seed based approach taught us to work with AI's strengths, not against its limitations. Validation and caching became critical.
3. Engineering practices: Writing tests first actually saved debugging time later. Git branches saved me when experiments failed. And documenting limitations honestly shows professionalism.

QUESTIONS?

